

XCAGI 企业 AI 员工平台

软件说明书

8. 核心架构与神经域

8.1 Neuro-DDD 分层

后端按 DDD（领域驱动设计）四层架构：interfaces（接口层）、application（应用层）、domain（领域层）、infrastructure（基础设施层）。

- interfaces：FastAPI 路由 + WebSocket + 桌面端 IPC 入口
- application：用例编排、事务管理、跨域协调
- domain：纯领域模型 + 领域服务，零外部依赖
- infrastructure：数据库、缓存、外部 SDK、MOD 加载

8.2 11 个神经域

系统把业务切成 11 个自治的「神经域」，每个域独立部署、独立扩缩容，通过 NeuroBus 消息总线互联：

1. account：账号、身份、Token 钱包
2. workspace：工作区、成员、权限
3. mod：MOD 商店、安装、生命周期
4. ai：AI 员工、模型路由、知识库（RAG）

- 5. intent：意图识别、对话状态、神经反射弧
- 6. workflow：可视化编排、自然语言编排
- 7. datasource：数据源适配、Excel 解析、ETL
- 8. approval：审批流、流程引擎、签批
- 9. notification：通知中心、邮件、IM 推送
- 10. integration：ERP/CRM/OA 对接、Webhook
- 11. audit：审计日志、合规报表、变更追踪

8.3 NeuroBus 神经总线

总线是事件驱动 + RPC 混合模式：域间通讯用事件发布订阅，跨域事务用 Saga 编排补偿。

消息带 trace_id 串联整条调用链，方便排查慢请求和故障定位。

每个域对外只暴露事件 schema，不暴露内部表结构，新增/下线域不会影响其他域。

9. MOD 生态详解

9.1 MOD 是什么

MOD = 一段可插拔的 Python 包，按统一接口注册能力到宿主。每个 MOD 包含：

- manifest.yaml：版本、依赖、权限、签名
- mod_main.py：入口点，提供 router、events、tools
- frontend 目录：可选，宿主会按需注入到前端菜单
- migrations/：数据库迁移脚本

9.2 三个官方 MOD

- 1. 出货 MOD：订单创建、库存扣减、物流回执

2. 客户 MOD：客户档案、联系人、商机跟进

3. 客服 MOD：工单、知识库、SLA 监控

这三个 MOD 都是开箱即用的小型业务系统，企业可以基于它们魔改。

9.3 私有 MOD 发布

企业可以发布只对自己可见的私有 MOD，比如对接自己公司系统的魔改版本。

私有 MOD 走内部签名通道，不进入公开商店，但可以复制一份到其他工作区复用。

9.4 MOD 沙箱

MOD 跑在受限沙箱里：访问文件、网络、数据库都要走权限白名单。

manifest 里声明需要什么权限，安装时用户确认，宿主按最小授权原则发放 token。

10. AI 员工编排

10.1 员工的概念

AI 员工 = 人格 + 技能 + 知识库 + 可用工具。

和人一样，员工有「岗位」（销售员、客服员、审计员）、「人设」（说话风格、专业度）、「技能」（能调的工具）。

10.2 员工的三种来源

1. 平台官方员工：商店里卖的训练好的员工，订阅即用

2. 自定义员工：自己配置人设、技能、知识库

3. 魔改员工：拿官方员工改 prompt、换模型、加新技能

10.3 多员工协同

复杂任务可调度多个员工分工完成。比如「分析上周销售并出报告」：

- 数据员负责从数据库抓数据
- 分析员负责做趋势分析
- 写手员负责生成可读报告

协同由 Supervisor 编排，自动合并各员工的结果。

10.4 知识库与 RAG

员工可以挂一个或多个知识库：上传 PDF/Word/Excel，系统自动切片、向量化、入库。

问答时按相关性召回最匹配的片段，喂给大模型。

向量化用 pgvector，存 PostgreSQL，不依赖额外服务。

11. 数据流与典型业务场景

11.1 一句话：数据怎么走的

「用户输入」→ 意图识别 → 反射弧/大模型 → 工具调用 → 域事件 → 数据持久化 → 反馈用户

每一步都有 trace 记录，挂了能在 ChatDebug 里看到。

11.2 场景 A：用户上传 Excel 自动入库

1. 用户拖一个 Excel 进对话
2. datasource 域识别表头、推断字段类型
3. 弹确认框给用户看「这是不是要导入到客户表」
4. 用户点确认后，生成导入规则并执行
5. 完成后告诉用户「已导入 N 条，跳过 M 条（原因）」

11.3 场景 B：销售员工处理一笔订单

1. 用户：「给百木鼎建一笔 5 桶木蜡油的订单」
2. 意图识别：调意图 = 创建订单，参数 = {客户:百木鼎, 商品:木蜡油, 数量:5}
3. 销售员工查商品库、查客户档案、查库存
4. 自动算价（含客户折扣）
5. 创建订单 → 扣库存 → 发通知给客户
6. 全程 < 3 秒

11.4 场景 C：多员工协同出报告

1. 用户：「分析上月销售并出份 PPT 报告」
2. Supervisor 拆任务：数据分析 + 报告撰写
3. 数据员抓数据、计算关键指标
4. PPT 写手员基于指标生成 PPT 大纲和文案
5. 设计师员配图、配版式
6. Supervisor 合并成一份 PPT，存到工作区

12. 错误处理与可靠性

12.1 神经总线的 8 大保险

- 去重 (Deduplicator)：相同消息不会重复处理
- 限流 (RateLimiter)：单个用户/工作区每秒最多多少请求
- 熔断 (CircuitBreaker)：下游服务挂了就快速失败
- 降级 (Degradation)：关键路径失败时给降级方案
- 追踪 (Tracer)：每次调用都有 trace_id 串联
- 审计 (Auditor)：关键操作写不可变日志

- 补偿 (Compensator) : 分布式事务的回滚机制
- 容错 (FaultTolerance) : 单点故障不影响全局

12.2 桌面端崩溃恢复

桌面客户端崩溃后下次启动会自动恢复：

- 未保存的对话草稿从本地 SQLite 拉回
- 上传中断的文件续传
- MOD 状态检查，不一致的自动重装

12.3 数据安全

- 传输全程 TLS
- 敏感字段入库前加密 (AES-256)
- 凭据存 Hashicorp Vault 风格的本地保险箱
- 关键操作有二次确认 (删除、批量导入、批量审批)
- 工作区之间数据完全隔离

12.4 监控与运维

自带「运维仪表盘」页面，实时看：

- 各域 QPS、错误率、P99 延迟
- AI 调用 token 用量、按员工/MOD 维度
- MOD 商店下载/安装/卸载趋势
- 关键业务事件流 (订单创建、审批通过等)

12.5 合规与审计

所有写操作都进审计表，admin 可以查询：

- 谁在什么时候改了什么
- 关键决策（比如审批通过）的原因
- 异常操作告警（异地登录、非工作时间大量删除等）

审计数据可导出 PDF/Excel 留档。

13. 数据模型与关键表

13.1 数据库选型

主存储用 PostgreSQL 16，启用 pgvector 扩展存向量。

缓存和分布式锁用 Redis 7.0；会话和临时草稿用 Redis；审计日志用 PostgreSQL（长期可追溯）。

桌面端本地用 SQLite 存草稿、缓存、配置。

13.2 关键表（按域）

account 域：users、identities、tokens、token_ledger

workspace 域：workspaces、members、roles、permissions

mod 域：mods、mod_versions、installations、subscriptions

ai 域：agents、agent_skills、knowledge_docs、embeddings (pgvector)

intent 域：intents、reflex_patterns、conversation_sessions

workflow 域：workflows、workflow_steps、workflow_runs

datasource 域：data_sources、schemas、import_rules

approval 域：approval_templates、approval_instances、approval_actions

audit 域：audit_logs（不可变）、change_history

13.3 通用字段

每张业务表都带：id (UUID) 、created_at、updated_at、created_by、tenant_id (工作区隔离)

软删除用 deleted_at，不用物理 DELETE，审计更友好。

14. API 接口规范

14.1 风格

RESTful + JSON over HTTPS；URL 资源用复数名词；动词用 HTTP Method。

返回统一信封：{code, data, message, trace_id}。错误码按域前缀 (ACCT/WORK/MOD/AI...)。

14.2 鉴权

所有非公开接口都要 Bearer Token。Token 来自登录后由 account 域签发。

WebSocket 长连接也用 Token 鉴权，握手时塞 query string。

14.3 几个核心接口

- POST /api/v1/auth/login：登录拿 Token
- POST /api/v1/chat/completions：与 AI 员工对话（流式）
- GET /api/v1/mods：列出可用 MOD
- POST /api/v1/mods/{id}/install：安装 MOD
- GET /api/v1/workspaces/{id}/members：查工作区成员
- POST /api/v1/datasources/{id}/import：触发数据导入

MOD 自带接口按 /api/v1/mods/{mod_name}/... 前缀注册，不冲突。

14.4 限流与配额

默认每个用户 60 次/分钟；每个工作区 600 次/分钟。

AI 对话按 token 配额计费，超额返回 429 + 充值链接。

15. 前端视图与交互

15.1 主要页面（按角色）

普通员工：工作台、聊天、MOD 入口、消息中心、个人设置

工作区管理员：成员管理、权限配置、Token 充值、操作审计

平台运营：MOD 商店、员工商店、计费管理、用户管理

系统管理员：域配置、模型路由、监控仪表盘、应急开关

15.2 核心视图详解

EmployeeWorkspaceView（员工工作台）：

首页布局，左侧栏是聊天 / MOD 切换，中间是工作区卡片，右上角是员工头像和待办。

支持深色模式、紧凑模式、无障碍（键盘 + 屏幕阅读器）。

ChatView（聊天视图）：

左下角是对话输入框（支持文本/语音/截图/文件），主区域是消息流，顶部是当前员工切换器。

AI 思考中显示「正在思考」+ 工具调用进度；可中断；可回滚到任意步骤。

AI EcosystemView（AI 生态视图）：

用 D3 力导向图展示员工之间、员工与 MOD 之间的关系，节点可拖动、可筛选、可点击进详情。

这个图是动态生成的，反映当前工作区实际安装的 MOD 和雇佣的员工。

15.3 组件库

基于 Vue 3 + TypeScript + Vite 自研一套轻量组件库（不引第三方 UI 框架）：

- 表单：FInput、FSelect、FDatePicker、FFileUpload
- 数据：FTable（虚拟滚动）、FPagination、FFilter
- 反馈：FToast、FModal、FConfirm、FLoading
- 业务：FEmployeeCard、FModCard、FTagPrinter、FFlowEditor

样式用 SCSS 变量 + CSS 变量双层主题，亮/暗/护眼/紧凑四套。

15.4 状态管理

Pinia store 按域划分：useAuth、useWorkspace、useMod、useAgent、useChat、useNotification。

持久化用 pinia-plugin-persistedstate，自动写 localStorage（加密关键字段）。

16. 项目开发历程

16.1 起点

v1.x 时期是单一发货单系统，给一家家具漆料公司内部用，跑了一年多。

后来发现这种「AI +

业务编排」的模式可以套到其他行业，但每个行业又都有自己的特定需求。

于是决定把业务系统拆成 MOD 插件，宿主只做平台。

16.2 v3-v8 的探索

v3：把业务拆成 MOD 雏形，但接口不标准，加一个 MOD 要改主程序。

v4：引入 NeuroBus 概念，域间通讯用事件。

v5：桌面客户端出现，但桌面和 Web 各自维护一套。

v6：AI 员工模型引入，按角色分类。

v7：FastAPI 完全替代 Flask，统一 async 鉴权。

v8：桌面和 Web 共用同一份 Vue 前端 + 同一份后端，差异只在交付形态。

16.3 v9 的定型

v9 是当前稳定版，相对 v8 的关键变化：

- MOD SDK 正式对外开放，第三方可开发 MOD
- 引入 Token 钱包 + 商业化体系
- 多员工协同的 Supervisor 模型
- RAG 知识库 + pgvector 集成
- NeuroBus 8 大可靠性机制正式落地

17. 创新点与技术亮点

17.1 三段式架构

「宿主 + MOD + AI 员工」三段式：宿主做平台、MOD 做业务、AI 员工做执行。

这三者通过统一接口解耦，可以独立演进、按需组合。

类似 SaaS + PaaS + 智能体，但跑在同一进程内，启动延迟 < 2 秒。

17.2 双形态交付

桌面 + Web 共用同一份前端 + 后端 + 数据，仅交付形态不同。

桌面端离线可用、本地 AI 模型跑得快；Web 端多人协作、统一升级。

用户按需切换，开发只维护一份代码。

17.3 神经反射弧

预定义高频指令走规则引擎 (<1ms) ，不确定的才扔给大模型。

实测：常见操作（查订单、新建客户）平均响应 8ms，比纯 LLM 路径快 50-100 倍。

省 token 也省 GPU，规模化后边际成本极低。

17.4 自然语言即编排

用户说一句「明天 9 点前把没付款的订单都提醒一遍」就自动生成定时 workflow。

不用学流程图编辑器，对非技术用户友好。

17.5 神经域自治

11 个域互不直接依赖，通过事件总线协作，单域升级不影响其他域。

某个 MOD 崩溃不会拖垮宿主（沙箱 + 健康检查），用户感知到的只是「该 MOD 暂时不可用」。

17.6 知识库按员工隔离

销售员的知识库和审计员的完全隔离，问 A 不会拿到 B 的数据。

权限细到员工级，满足企业合规要求。

18. 后续规划

18.1 短期 (v9.x)

- MOD 商店正式对外开放，发布平台开发文档
- AI 员工市场 MVP，开放第三方员工上架
- 移动端 H5 适配（响应式 + 触屏优化）
- 多语言（中/英/日/越）

18.2 中期 (v10)

- 工作流可视化编辑器（拖拽式）
- 自托管大模型一键部署（Ollama 集成）
- 跨工作区数据联邦（多个工作区共享部分数据）
- 移动端原生 App（iOS / Android）

18.3 长期（v11+）

- 跨平台边缘部署（ARM 服务器 + K3s）
- 多模态员工（语音、视频、3D）
- 行业大模型微调（按行业出预训练模型）
- 员工协作市场（员工之间互相调用、付费）

19. 开发者与团队

19.1 主要开发者

本软件由独立开发者李佳洧设计并实现。

开发者从 v1 时期独自维护整套代码到 v9，过程中根据实际业务反馈反复重构。

所有架构决策（Neuro-DDD、神经反射弧、MOD 沙箱、Token 钱包）均出自一手需求，

没有走「先抄开源再魔改」的路线。

19.2 参与方式

项目代码仓库在 GitHub：github.com/42433422/ai-excel-helper（开发期）

欢迎提交 Issue、PR、测试用例。Bug 报告 7 天内回复，新功能 30 天内评估。

MOD / 员工开发请参考仓库里的 mod_sdk 文档和 agent_template 模板。

19.3 致谢

本项目使用了以下开源项目（部分）：

- 后端：FastAPI、SQLAlchemy、pgvector、Celery、Redis、uv
- 前端：Vue 3、Vite、Pinia、D3、Element Plus 借鉴
- AI：PyTorch、Transformers、PaddleOCR、sentence-transformers
- 桌面：Electron
- 工具：reportlab、pydantic、httpx、rich

所有依赖都使用 Apache 2.0 / MIT / BSD 等宽松许可证。

20. 与同类产品对比

20.1 vs 传统 ERP

- 实施周期：XCAGI 几小时上线，ERP 几个月
- 定制能力：XCAGI 装 MOD 即可，ERP 要改源码
- 学习成本：XCAGI 用聊天交互，ERP 要学专门操作
- 缺点：XCAGI 不适合流程固定、规模大的企业（金融、大型制造）

20.2 vs 通用 ChatGPT / Claude

- 数据隔离：XCAGI 数据在自己服务器，ChatGPT 数据在 OpenAI
- 业务动作：XCAGI 能直接调业务系统，ChatGPT 只能回答
- 模型选择：XCAGI 可自托管开源模型，ChatGPT 只能 OpenAI
- 缺点：通用大模型的语言能力比 XCAGI 自带员工强

20.3 vs 低代码平台（宜搭、明道云）

- 智能能力：XCAGI 内置 AI 员工，低代码要自己接

- 开放性：XCAGI 完全开源 + MOD 可扩展，低代码多闭源
- 学习曲线：XCAGI 偏程序员友好，低代码偏业务人员友好

21. 用户体验与反馈

21.1 交互原则

- 默认安全：危险操作都要二次确认
- 可逆优先：能撤销的就别让用户重做
- 即时反馈：每个操作 < 100ms 给视觉反馈
- 渐进披露：常用功能放第一屏，长尾功能藏菜单

21.2 反馈渠道

- 内置反馈：菜单 → 帮助 → 反馈建议
- GitHub Issue：技术问题、Bug 报告
- 邮件：xcagi-support@example.com（正式渠道）
- 用户群：扫码进 QQ/微信群（首页底部）

21.3 版本更新策略

- 大版本（v10、v11）：每年 1-2 次，可能改架构
- 小版本（v9.1、v9.2）：每月 1-2 次，加功能
- 补丁（v9.0.1）：周更，修 Bug
- LTS：每 4 个大版本出一个 LTS，社区支持 2 年

22. 典型案例

22.1 案例 A：家具漆料贸易公司

客户：川内一家家具漆料贸易商，10 人小公司，每天 50-100 单。

痛点：发货单靠 Excel 抄写，月底对账要 3 天。

方案：装上「出货 MOD」+ 雇佣「销售员工」+「对账员工」。

效果：发货单录入从 5 分钟/单降到 30 秒，月底对账 1 小时出报表。

ROI：3 个月回本。

22.2 案例 B：连锁餐饮品牌

客户：川内一家 20 家门店的连锁餐饮。

痛点：总部和门店之间数据不同步，物料损耗对不上。

方案：装「客户 MOD」+ 魔改「出货 MOD」对接门店系统 + 雇佣「审计员」。

效果：物料损耗率从 8% 降到 3%，总部对账从 2 周缩到 1 天。

22.3 案例 C：财税服务公司

客户：一家 50 人财税代理公司，服务 200+ 小微企业。

痛点：每月要处理几千张发票，人工录入慢且错。

方案：自建私有 MOD「发票处理」+ 知识库（税务法规）+ 雇佣「录入员」。

效果：发票处理效率提升 5 倍，错误率 < 0.5%。

23. 部署与运维建议

23.1 推荐的部署架构

- 50 人以下：单机部署，1 台 8 核 16GB 服务器够了
- 50-200 人：双机热备，前端负载均衡 + 后端双实例
- 200 人以上：K8s 集群 + PostgreSQL 主从 + Redis Sentinel

- 大模型推理：单独 GPU 节点，不和主业务混部

23.2 备份策略

- 数据库：每日全量 + 实时 WAL 归档，保留 30 天
- 文件：每日同步到对象存储，保留 90 天
- 配置：Git 化管理，每次变更打 tag
- 灾难演练：每季度一次恢复演练

23.3 监控指标

- 业务：日活、订单量、AI 调用次数、Token 消耗
- 系统：CPU、内存、磁盘 IO、网络带宽
- 应用：QPS、P99 延迟、错误率、慢请求占比
- 业务告警：订单异常下降、AI 失败率飙升、Token 余额不足

24. 术语表

24.1 核心概念

- 宿主 (Host)：XCAGI 平台本身，提供 MOD 加载、AI 编排、数据管理等基础能力
- MOD (Module)：业务模块，宿主上按需安装，如出货 MOD、客户 MOD
- AI 员工 (Agent)：可对话、可调工具的智能体，按角色分类
- 工作区 (Workspace)：多租户隔离单位，对应一家公司或一个项目
- 神经域 (Neural Domain)：后端的 11 个自治服务域
- NeuroBus：神经域之间的消息总线
- 神经反射弧：高频指令的快速匹配机制

- Token 钱包：AI 调用的计费账户

24.2 技术名词

- Neuro-DDD：神经领域驱动设计，本项目自创的分层架构
- FastAPI：Python 异步 Web 框架，本项目后端基础
- pgvector：PostgreSQL 的向量扩展，本项目用于 RAG
- RAG：检索增强生成，给大模型喂相关文档片段以提升回答质量
- LLM：大语言模型，本项目支持 OpenAI/DeepSeek/通义千问等
- Electron：桌面应用壳，本项目桌面客户端基于此
- Pinia：Vue 3 的状态管理库
- Saga：分布式事务编排模式，本项目 NeuroBus 跨域事务用此实现

25. 常见疑问解答 (FAQ)

25.1 为什么不用 SaaS 模式？

SaaS 模式下数据全在云端，对很多企业（尤其传统行业）不放心。

XCAGI 强调数据本地化（桌面版）或自托管（Web 版），数据主权归用户。

代价是部署稍复杂，但换来合规性和灵活度。

25.2 大模型成本怎么控制？

神经反射弧先把高频指令走规则引擎（零成本），剩余才到大模型。

支持多家模型路由，按场景选最便宜的。

token 消耗按员工/MOD 维度统计，贵的环节用户能直接看到。

25.3 MOD 之间会不会互相干扰？

MOD 跑在沙箱里，资源、网络、数据库访问都受 manifest 限制。

MOD 之间通讯走 NeuroBus 事件总线，要先声明依赖和事件 schema。

某个 MOD 崩溃只会影响自己的功能，不会拖垮宿主。

25.4 私有大模型怎么接？

v10 起计划集成 Ollama，本地一键起模型服务，XCAGI 自动发现并注册。

v9.x 需要自己改 ai 域的 model_router 配置，加个新 provider。

推荐模型：Qwen2.5-7B、DeepSeek-V2-Lite、Llama-3-8B（中文都过得去）。

25.5 怎么从 v8 升到 v9？

v8 的 MOD 在 v9 不能直接用，因为 manifest 规范变了。

v9 提供了 v8→v9 迁移工具，能自动转 80% 的常见 MOD。

数据层兼容 v8，schema 没动；只换了 API 鉴权方式，需要用户重新登录一次。

26. 合规与法律

26.1 数据合规

- 国内：符合《个人信息保护法》《数据安全法》
- 欧盟：符合 GDPR（如果企业有欧洲业务）
- 医疗：符合 HIPAA（如果要扩展医疗 MOD）
- 财务：符合等保 2.0 三级（如果要服务金融客户）

26.2 知识产权

- 本软件已申请软件著作权（登记号待补）
- 自带 MOD 的著作权归开发者所有

- 第三方 MOD 的著作权归开发者所有
- 用户数据归用户所有，平台不主张任何权利

26.3 免责声明

本软件按「现状」提供，不承诺无错误或适用于特定目的。

因使用本软件产生的业务损失，开发者不承担责任。

使用前请阅读完整的 LICENSE 文件。

27. 附录

27.1 附录 A：默认监听端口

- FastAPI 后端：8000
- PostgreSQL：5432
- Redis：6379
- pgvector：随 PostgreSQL，无需单独端口
- 桌面壳：跟宿主 FastAPI 共用

27.2 附录 B：环境变量一览

- DATABASE_URL：PostgreSQL 连接串
- REDIS_URL：Redis 连接串
- JWT_SECRET：JWT 签名密钥
- OPENAI_API_KEY：OpenAI 模型 key（可选）
- DEEPSEEK_API_KEY：DeepSeek 模型 key（可选）
- DEFAULT_LLM_PROVIDER：默认模型供应商

27.3 附录 C：升级路径

v9.0.x 补丁：直接覆盖部署，数据无影响

v9.x → v10.0：会有 schema 变更，需要运行迁移脚本

v8 → v9：先升级 v8 最后一个补丁到 v8.5，再用迁移工具

v1-v7 → v9：不支持直接升级，建议新部署+数据导入

27.4 附录 D：联系方式

- 项目主页：<https://xcagi.example.com>
- 文档站：<https://docs.xcagi.example.com>
- 代码仓库：<https://github.com/42433422/ai-excel-helper>
- 邮箱：xcagi-support@example.com
- 用户群：菜单 → 帮助 → 用户群（首页底部二维码）

版本：V10.0.0

开发完成日期：2026 年 5 月

著作权人：李佳洸

生成日期：2026年06月25日

目录

1. 软件概述

2. 软件功能

3. 技术特点
4. 运行环境
5. 安装说明
6. 使用说明
7. 常见问题
8. 核心架构与神经域
9. MOD 生态详解
10. AI 员工编排
11. 数据流与典型业务场景
12. 错误处理与可靠性
13. 数据模型与关键表
14. API 接口规范
15. 前端视图与交互
16. 项目开发历程
17. 创新点与技术亮点
18. 后续规划
19. 开发者与团队
20. 与同类产品对比

21. 用户体验与反馈

22. 典型案例

23. 部署与运维建议

24. 术语表

25. 常见疑问解答 (FAQ)

26. 合规与法律

27. 附录

1. 软件概述

XCAGI 企业 AI 员工平台是一款跨平台的企业级 AI 员工桌面平台。

它由「宿主（空壳）+ 行业 MOD（业务模块）+ AI 员工（智能体）」三部分组成：

- 宿主：通用的桌面 + Web 双形态壳子，所有客户部署的都是同一份空壳
- MOD：从平台商店按需安装的业务模块，宿主装上「出货 MOD」就是出货系统，装上「客服 MOD」就是客服系统
- AI 员工：可编排、可替换的智能体（人设、技能、知识库），完成实际业务动作

1.1 软件定位

v9.0 不再是单一的发货单系统，而是一个「可组装的企业 AI 员工平台」。

面向想把 AI 用到实际业务里、但又不想每个场景都重写一遍后端的小中型企业。

形态上同时支持 Windows / macOS 桌面客户端和 Web 自托管部署，按需选。

1.2 主要用途

- 通用业务编排：聊天 + 自然语言指令驱动业务动作，不限具体行业
- MOD 商店：按需启用 ERP / 出货 / 客服 / 商机 / 审批等模块
- AI 员工市场：可购买/雇佣训练好的智能体（销售员、客服员、审计员等）
- Token 钱包：按调用量计费，支持个人 / 企业两种钱包
- 跨端使用：桌面客户端、Web 浏览器、Docker 自托管三种交付形态并存

2. 软件功能

2.1 AI 员工工作台

左侧是对话框，可调用挂载的 AI 员工。员工按角色分类（销售、客服、运营等），

每个员工有自己的人格、技能、可用工具和知识库。

支持多员工协同：一个指令可以同时调度多个员工分工完成（比如「查一下这个月销售额并出报告」）。

2.2 MOD 商店与员工商店

- MOD 商店：业务模块市场（出货、库存、合同、CRM、审批、数据源等），按订阅计费
- 员工商店：训练好的 AI 员工（人设 + 技能 + 知识库），按调用次数计费
- 商店里的 MOD / 员工都经过 manifest 校验和签名验证
- 企业可发布私有 MOD / 员工给内部使用

2.3 业务编排

- 自然语言编排：说一句话自动拆解成可执行步骤
- 可视化编排：拖拽式搭建工作流（触发器 + 条件 + 动作）
- Excel / 数据库导入即编排：上传一份 Excel 自动识别表头、生成导入规则
- 跨系统对接：通过数据源适配器连 ERP / CRM / OA / 数据库 / API

2.4 Token 钱包

- 个人 / 企业两种钱包，按调用 AI 的 token 数结算
- 支持充值、提现、对公转账、发票管理
- 用量统计：按员工 / 按 MOD / 按时间段维度看消耗
- 软著只关心技术实现，不涉及具体计费策略

2.5 数据源与对接

- 关系型数据库：PostgreSQL / MySQL / SQL Server / SQLite
- NoSQL：Redis / MongoDB（只读视图）
- API：RESTful / GraphQL / Webhook
- 文件：Excel / CSV / PDF（OCR 解析）

2.6 审批与流程

- 自定义审批流：条件分支 / 并行 / 串行 / 委派
- 审批中心：待办 / 已办 / 我发起，三栏视图
- 移动端审批：手机浏览器兼容，触屏友好

2.7 双形态交付

- 桌面客户端：Electron 壳 + 内嵌 FastAPI 子进程 + 复用 Vue 3 前端
- Web 部署：Docker / Nginx 反代 + FastAPI + 同一份前端
- 两种形态共用一套数据、一套账号、一套商业化能力

3. 技术特点

3.1 Neuro-DDD 架构

后端按 DDD 分层（application / domain / infrastructure），并把 AI 对话与工作流的用例编排单独抽出来。

11 个自治神经域（订单、出货、合同、审批、AI、Token、MOD、集成等）通过 NeuroBus 互联，

域之间互不知道实现细节，只通过总线消息和事件通讯。

3.2 MOD 插件生态

- 第三方 MOD 通过 SDK 接入，必须提供 manifest.yaml（版本、权限、依赖、签名）
- 宿主启动时扫描 /mods 目录，做签名校验和依赖图检查
- MOD 商店提供版本管理、灰度发布、回滚

3.3 AI 能力

- 混合意图识别：BERT 做语义、DeepSeek 大模型做复杂推理、RASA 管对话状态
- 神经反射弧：常见指令<1ms 反射回来，不绕大模型
- 知识库：基于 pgvector 的 RAG，按员工粒度隔离
- 多模型适配：支持 OpenAI / DeepSeek / 通义千问 / 自托管开源模型

3.4 可靠性机制

NeuroBus 集成 8 大可靠性保障：去重、限流、熔断、降级、追踪、审计、补偿、容错。

FastAPI 端用 Redis 分布式锁 + async 装饰器统一认证与会话。

CI 强制 Bandit / Safety / Flake8 硬失败，覆盖率门禁 60%。

3.5 技术栈

后端	Python 3.11 + FastAPI 0.109 + SQLAlchemy 2.0 + Celery 5.3
----	---

前端	Vue 3.4 + TypeScript 5.5 + Vite 5.0 + Pinia
桌面壳	Electron + 内嵌 FastAPI 子进程
数据库	PostgreSQL 16 (含 pgvector) + Redis 7.0
AI/ML	PyTorch 2.0 + Transformers 4.35 + pgvector + OpenAI SDK
部署	Docker + Docker Compose + Nginx

4. 运行环境

4.1 硬件最低要求（桌面端）

- CPU：x86_64 / arm64，4 核及以上
- 内存：8GB 起步，跑大模型建议 16GB+
- 硬盘：10GB 可用空间（不含数据）
- 显示器：1080p
- 网络：联网用 AI 员工（可配本地模型离线运行）

4.2 硬件最低要求（Web 部署 / 服务端）

- CPU：8 核及以上
- 内存：16GB 起步，32GB+ 推荐
- 硬盘：100GB SSD（数据 + 模型）
- 网络：10Mbps 带宽

4.3 软件依赖

- 系统：Windows 10/11、macOS 12+、Ubuntu 20.04/22.04
- Python 3.11+
- Node.js 18+（桌面壳构建用）

- PostgreSQL 16（必装，启用 pgvector 扩展）
- Redis 7.0+
- 浏览器：Chrome / Edge / Firefox 最新版

5. 安装说明

5.1 桌面客户端（普通用户）

1. 到 <https://github.com/42433422/ai-excel-helper/releases> 下载对应系统的安装包
2. Windows 跑 .exe，macOS 拖进 Applications，Linux 用 .ApplImage
3. 首次启动会引导注册账号、实名、创建工作区
4. 工作区创建后进 MOD 商店装第一个 MOD（比如「出货」）就能用了

5.2 Web 部署（管理员）

1. 装 Docker 24+ 和 Docker Compose
2. 拉代码：git clone <https://github.com/42433422/ai-excel-helper.git>
3. 改 docker-compose.yml 里的数据库密码和域名
4. 起服务：docker compose up -d
5. 反代 Nginx 配 SSL 证书（Let's Encrypt 即可）
6. 浏览器开 <https://your-domain> 就能用了

5.3 开发模式

1. 装 Python 3.11、Node.js 18+、uv 包管理
2. 装依赖：uv sync（推荐）或 pip install -r requirements.txt
3. 启动后端：python app/bootstrap.py run

4. 启动前端：cd frontend && npm run dev

5. 浏览器开 <http://localhost:5173> 即可

6. 使用说明

6.1 注册和登录

桌面端第一次打开会引导注册；Web 端进首页点「注册」按提示走。

登录用邮箱 + 密码，或者选第三方登录（GitHub / Google / 微信，按地域开）。

6.2 创建工作区

登录后第一步是建工作区：工作区 = 一家公司 / 一个项目，

工作区之间数据隔离，账号可以同时属于多个工作区。

6.3 安装 MOD

1. 进「MOD 商店」

2. 选需要的 MOD（比如「出货」「客户」「AI 员工：销售员」）

3. 点安装，确认权限

4. 安装后左侧菜单会出现对应功能入口

6.4 用 AI 员工干活

1. 左侧对话框选一个员工（已雇佣的）

2. 直接打字，比如「把上周的销售数据出份报告」

3. 员工拆解指令、调工具、给结果

4. 复杂任务可挂后台，做完会推消息

6.5 数据源对接

1. 进「数据源」
2. 选类型（数据库 / API / 文件），填连接信息
3. 系统会探测结构，生成可读视图
4. AI 员工就能基于这些数据回答问题

7. 常见问题

7.1 桌面版打不开？

先看 macOS 是不是被 Gatekeeper 拦了——系统设置 → 隐私与安全 → 仍要打开。

Windows 上看杀毒软件有没有误报，加白名单再试。

7.2 AI 员工没回应？

1. 检查 Token 钱包余额
2. 看网络能不能通 OpenAI / DeepSeek
3. 进「ChatDebug」页面看请求日志

7.3 MOD 装不上？

通常是依赖冲突或者签名校验失败。删掉 /mods 下旧版本，看日志里报的具体错。

7.4 数据怎么备份？

PostgreSQL 用 pg_dump 周期备份，应用层有「数据导出」功能可以倒出关键表。

桌面版的数据默认在 ~/Library/Application Support/XCAGI 目录，备份这个目录就行。

XCAGI 企业 AI 员工平台 软件说明书

著作权人：李佳洸 版本：V10.0.0

生成时间：2026年06月25日 03:38:53